Instituto Tecnológico Superior de Xalapa

INSTITUTO TECNOLÓGICO SUPERIOR DE XALAPA

ING. EN SISTEMAS COMPUTACIONALES

3° Semestre | Grupo: B

ESTRUCTURA DE DATOS

VAZQUES HERRERA VICTOR HUGO

UNIDAD 5
TAREA 2

PRESENTA

257O00081
Brayan Mejia Mora

22 DE NOVIEMBRE DEL 2026

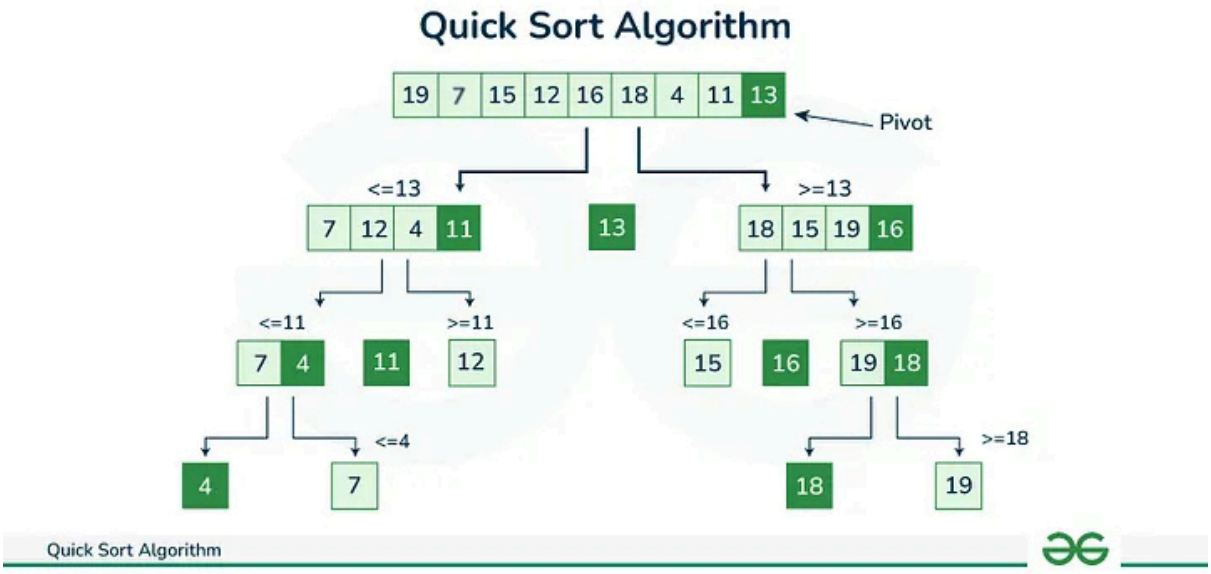
QuickSort vs ShellSort

Ubicación del código

El siguiente link corresponde justamente a la ruta (repositorio público de GitHub) donde se encuentra la carpeta de QuickSort y ShellSort; cada una en su propia código (implementado en java) y su respectivo archivo README.md:

https://github.com/brianlewyn/data-structure-tec-3th-semester/tree/main/Unidad5

Quick Sort



¿Qué es?

Quick Sort es un algoritmo de ordenamiento basado en la estrategia de divide y vencerás. Su operación central es la partición, que consiste en:

1. **Elegir un pivote**: seleccionar un elemento del arreglo como referencia.
2. **Reorganizar el arreglo**: ubicar todos los elementos menores que el pivote a la izquierda y todos los mayores a la derecha. El pivote termina en su posición correcta dentro del ordenamiento.
3. **Aplicar recursivamente**: repetir el mismo proceso en los subarreglos izquierdo y derecho.

Después de cada partición, el pivote queda en su posición final y nunca vuelve a moverse. Cada llamada recursiva coloca más pivotes en sus posiciones correctas hasta que todo el arreglo queda ordenado.

¿Cómo funciona?

El algoritmo tiene dos componentes principales: **la operación de partición** y **el controlador recursivo**.

La operación de partición

La partición es el corazón de Quick Sort. Existen dos esquemas de partición clásicos:

Esquema de partición de Lomuto (más simple, más fácil de entender):

- Usa el último elemento como pivote
- Mantiene un índice de frontera i que rastrea dónde termina la región de "menores que el pivote"
- Recorre de izquierda a derecha, intercambiando los elementos menores que el pivote hacia el frente
- Al final, intercambia el pivote a su posición correcta

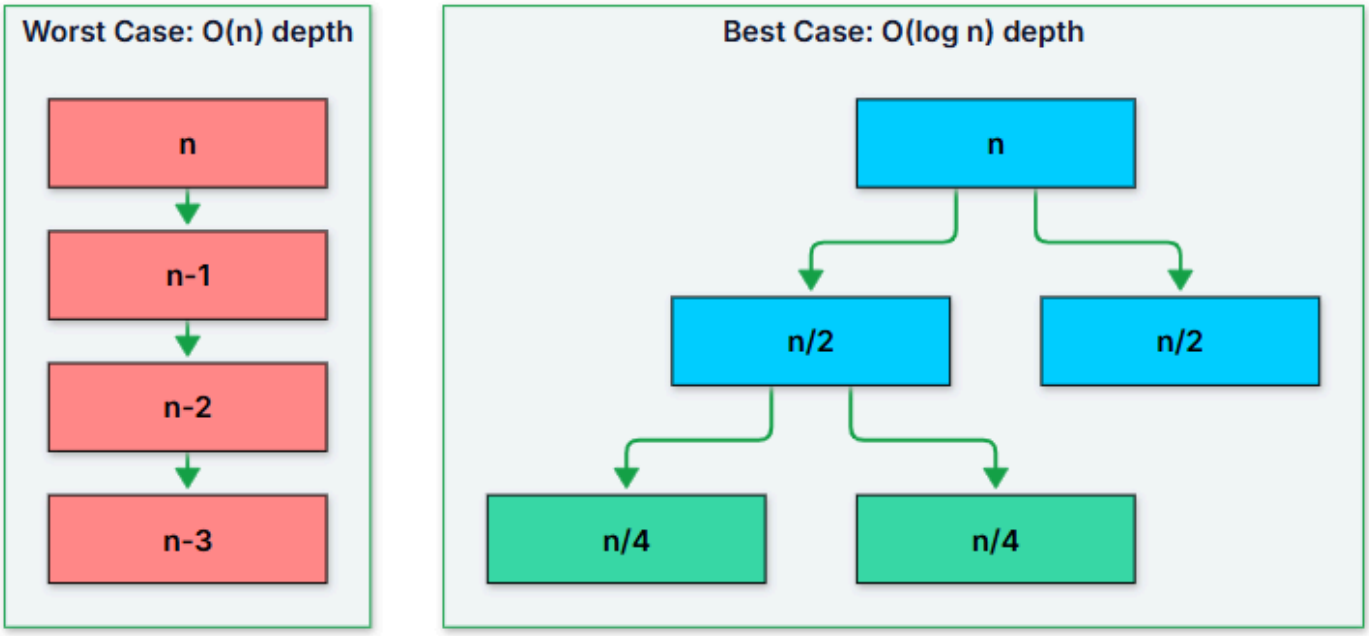
Esquema de partición de Hoare (más eficiente):

- Usa el primer elemento como pivote
- Usa dos punteros que comienzan desde extremos opuestos
- Los punteros se mueven uno hacia el otro, intercambiando los elementos que están en el lado incorrecto
- Realiza menos intercambios en promedio (aproximadamente tres veces menos que Lomuto)

Selección del esquema

La implementación presentada en este reporte usa el esquema de Lomuto, porque es más fácil de seguir y menos propensa a errores al implementarla. El esquema de Hoare realiza menos intercambios, pero es más difícil de implementar correctamente.

Análisis de Complejidad



Tiempo: O(n log n) en promedio

Cada partición hace O(n) trabajo (recorre el subarreglo una vez). La clave es la profundidad de recursión:

- Si el pivote divide el arreglo a la mitad, la profundidad es log n.
- Cada nivel suma O(n) trabajo entre todas sus particiones.
- Total: O(n) · O(log n) = O(n log n).

Tiempo: O(n²) en el peor caso

Ocurre cuando la partición es máximamente desbalanceada: el pivote es siempre el menor o el mayor, una partición recibe n−1 elementos y la otra 0. Esto genera n niveles en lugar de log n, con O(n) trabajo por nivel: O(n) · O(n) = O(n²).

¿Cuándo? El caso clásico: arreglo ya ordenado (ascendente o descendente) eligiendo siempre el primer o último elemento como pivote.

Espacio

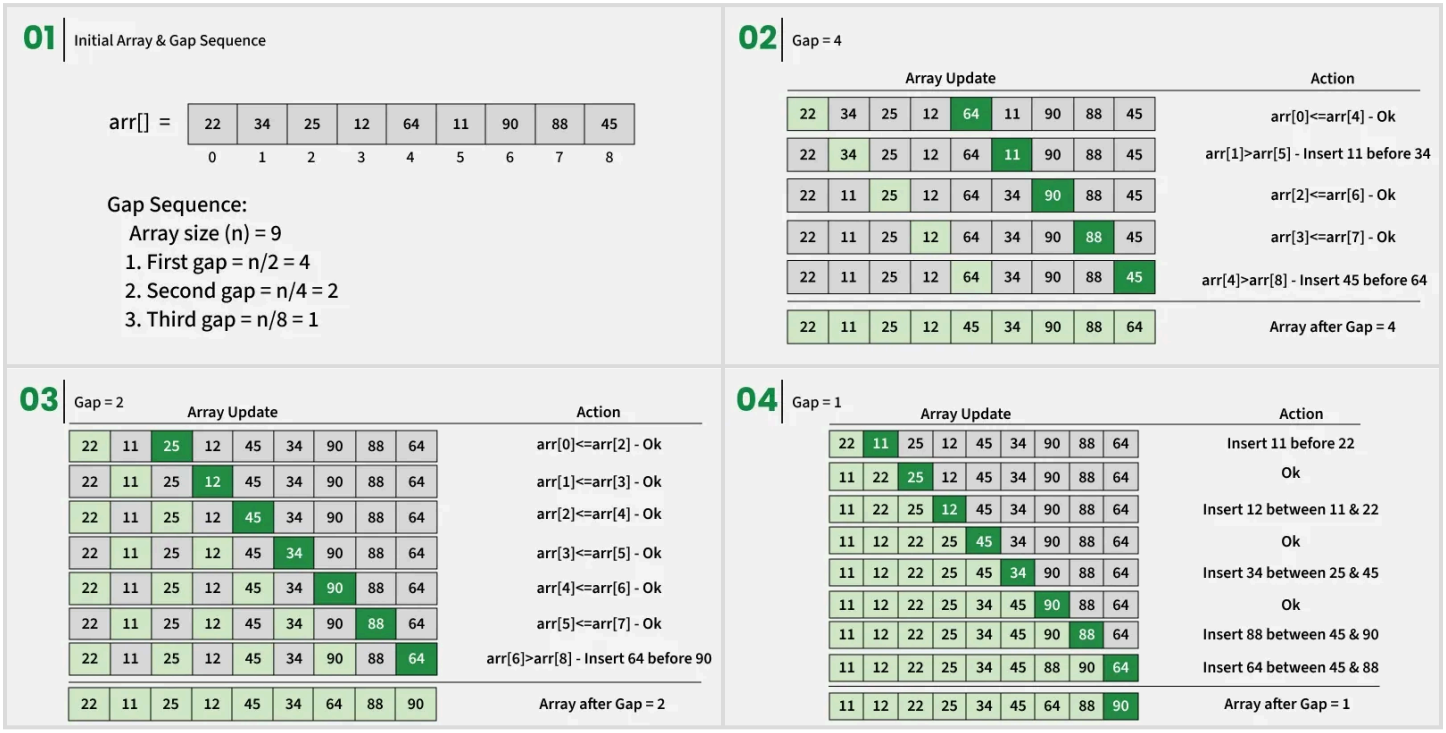
- Ordena in-place: no asigna arreglos extra.
- Usa la pila de llamadas: profundidad O(log n) en el mejor caso y O(n) en el peor (riesgo de stack overflow en arreglos ordenados muy grandes).

Estabilidad

No es estable. La partición intercambia elementos a larga distancia, y un elemento igual puede pasar por delante de otro igual que aparecía antes.

Ejemplo: [3a, 1, 3b, 2] con pivote 2 (Lomuto): [1, 2, 3b, 3a]. El orden 3a, 3b se invierte.

ShellSort



¿Qué es?

Shell Sort es una generalización del ordenamiento por inserción que intercambia elementos lejanos entre sí. En lugar de comparar solo vecinos adyacentes, compara elementos separados por una brecha (gap) y aplica inserción a cada grupo espaciado por esa brecha. La brecha se reduce en cada pasada hasta llegar a 1, donde se hace una inserción estándar sobre un arreglo ya casi ordenado.

¿Por qué funciona? Ordenar primero los elementos lejanos elimina el desorden grande rápidamente. Cada pasada con menor brecha acerca el arreglo al orden final. Cuando la brecha llega a 1, casi todo está en su sitio, así que la última pasada corre en casi O(n).

Diferencia clave con la inserción simple: en la inserción, cada elemento solo se compara con su vecino inmediato, por lo que moverse n posiciones cuesta n intercambios: $O(n^2)$. Shell Sort rompe esa restricción: los elementos de un mismo grupo se ordenan juntos y pueden saltar grandes distancias en un solo paso, reduciendo el total de operaciones.

¿Cómo funciona?

El algoritmo repite tres pasos en un bucle:

1. **Elegir una brecha** (normalmente la mitad del tamaño del arreglo).
2. **Ordenar por inserción** los elementos separados por esa brecha.
3. **Reducir la brecha** (a la mitad) y repetir hasta que sea 1.

Con brechas grandes, los elementos se agrupan por distancia y se ordenan dentro de cada grupo. Al reducir la brecha, los grupos se mezclan más. Cuando la brecha llega a 1, se hace una inserción normal, pero el arreglo ya está casi ordenado, así que es muy rápido.

Nota: La secuencia original de Shell (n/2) es la más simple y la más comúnmente enseñada, pero no es la más eficiente. Las secuencias de *Knuth* y *Sedgewick* rinden mejor en la práctica porque evitan la "interacción de incrementos", donde ciertos valores de brecha no logran comparar elementos que ya habían sido comparados en pasadas anteriores.

Análisis de Complejidad

Tiempo: O(n log n) en el mejor caso

En un arreglo ya ordenado, cada pasada hace una comparación por elemento sin desplazamientos.

Tiempo: O(n^2) en el peor caso

La secuencia original usa potencias de 2 (n/2, n/4, ..., 2, 1), y las posiciones pares e impares nunca comparten subarreglo hasta la pasada final (brecha = 1). Si los peores elementos se colocan explotando ese aislamiento, la última pasada equivale a una inserción completa sobre n elementos.

Espacio

Usa O(1) de espacio auxiliar. Ordena in-place usando solo una variable temporal para el intercambio

Estabilidad

No es estable. Elementos iguales pueden cambiar de orden relativo en las pasadas con brecha. Ejemplo: dos elementos iguales en posiciones 0 y 3 con brecha = 2 pertenecen a subarreglos distintos (0, 2, 4 vs. 1, 3) y podrían reordenarse entre sí.

Comparación

Property	Quick Sort	Shell Sort
Mejor caso	O(n log n)	O(n log n)
Caso promedio	O(n log n)	Empíricamente cercano a $O(n^{3/2})$, no demostrado
Peor caso	$O(n^2)$	$O(n^2)$
Espacio o memoria adicional	O(log n)	O(1)
Estabilidad	No	No
In-Place (ordena o modifica los datos directamente sobre la estructura original)	Yes	Yes
Recursivo	Yes	No

Pruebas en Consola

Pruebas con el mismo orden de elementos

Quick Sort

```
======= Ejemplo de Arreglo de Clase (n=8) =======

Arreglo original: [6, 1, 5, 8, 3, 2, 7, 4]
Arreglo ordenado: [1, 2, 3, 4, 5, 6, 7, 8]

Comparaciones:       15
Intercambios:        14
Llamadas recursivas: 11
```

Shell Sort

```
======= Ejemplo de Arreglo de Clase (n=8) =======

Arreglo original: [6, 1, 5, 8, 3, 2, 7, 4]
Arreglo ordenado: [1, 2, 3, 4, 5, 6, 7, 8]

Comparaciones:       22
Intercambios:        8
```

Prueba completa del Quick Sort

```
======= Ejemplo de Arreglo de Clase (n=8) =======

Arreglo original: [6, 1, 5, 8, 3, 2, 7, 4]
Arreglo ordenado: [1, 2, 3, 4, 5, 6, 7, 8]

Comparaciones:       15
Intercambios:        14
Llamadas recursivas: 11

======= Mejor Caso Teórico (n=10) =======

Arreglo original: [1, 3, 2, 5, 4, 8, 9, 7, 10, 6]
Arreglo ordenado: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Comparaciones:       19
Intercambios:        15
Llamadas recursivas: 11

======= Caso Promedio (Muestra Aleatoria) (n=10) =======

Arreglo original: [7, 3, 9, 1, 5, 8, 2, 6, 10, 4]
Arreglo ordenado: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Comparaciones:       19
Intercambios:        13
Llamadas recursivas: 11

======= Peor Caso Teórico (n=10) =======

Arreglo original: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Arreglo ordenado: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Comparaciones:       45
Intercambios:        54
Llamadas recursivas: 19
```

Prueba completa del Shell Sort

```
======= Ejemplo de Arreglo de Clase (n=8) =======

Arreglo original: [6, 1, 5, 8, 3, 2, 7, 4]
Arreglo ordenado: [1, 2, 3, 4, 5, 6, 7, 8]

Comparaciones:       22
Intercambios:        8

======= Mejor Caso Teórico (n=10) =======

Arreglo original: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Arreglo ordenado: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Comparaciones:       22
Intercambios:        0

======= Caso Promedio (Muestra Aleatoria) (n=10) =======

Arreglo original: [7, 3, 9, 1, 5, 8, 2, 10, 4, 6]
Arreglo ordenado: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Comparaciones:       32
Intercambios:        15

======= Peor caso teórico (n=10) =======

Arreglo original: [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
Arreglo ordenado: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Comparaciones:       27
Intercambios:        13
```

Conclusión

QuickSort es un algoritmo de divide y vencerás basado en la partición: elige un pivote, reorganiza los elementos a su alrededor y recurre en cada subarreglo. Es O(n log n) en promedio con particiones balanceadas, pero degrada a O(n²) cuando son desbalanceadas. Con Lomuto y pivote en el extremo, el peor caso son los arreglos ya ordenados. Ordena in-place, consume pila de llamadas y no es estable.

ShellSort generaliza el ordenamiento por inserción usando una brecha que se reduce hasta 1. Su mejor caso es el arreglo ya ordenado (0 desplazamientos), y su peor caso es O(n²) cuando los grandes quedan en posiciones pares y los pequeños en impares, ya que no se mezclan hasta gap=1. Es in-place (O(1) auxiliar) y tampoco es estable.

Lo que aprendí: que Big-O describe crecimiento, no valores exactos: ser O(n²) no es hacer 100 operaciones para n=10, sino cuadruplicar el costo al duplicar n (45 → 190 → 780), y oculta constantes como el ½ de n(n−1)/2 (45 en vez de 100). Que n log n es mucho menor que n² y la brecha crece con la escala (~3× a n=10, ~100× a n=1,000). Que el peor caso depende de la implementación (arreglo ordenado en QuickSort, patrón alternado en ShellSort) y que la escala importa: a n=10 los contrastes son sutiles, a n=16 o más se vuelven evidentes. Y que los contadores empíricos validan la teoría: el peor caso de QuickSort dio 45 = n(n−1)/2 para n=10, y en ShellSort el conteo creció con el tamaño y el patrón, confirmando ambas complejidades. La medición empírica verifica la teoría, aunque no reemplaza el análisis formal.

Bibliografía

GeeksforGeeks. (2025, 20 diciembre). *Shell Sort*. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/shell-sort/

Quick Sort — Visualizador de algoritmos. (2026, 21 julio). Algorithm Visualizer. https://alg0.dev/es/quick-sort/

Shell Sort — Visualizador de algoritmos. (2026, 21 julio). Algorithm Visualizer. https://alg0.dev/es/shell-sort/

Singh, A. P. (2026a, julio 10). *Shell Sort*. AlgoMaster.io - Master Software Engineering Interviews. https://algomaster.io/learn/dsa/shell-sort

Singh, A. P. (2026b, noviembre 10). *Quick sort*. AlgoMaster.io - Master Software Engineering Interviews. https://algomaster.io/learn/dsa/quick-sort

Hashemi, E. (2024, julio 3). *Sorting Algorithms — QuickSort explained*. Medium. https://medium.com/@el.elhamhashemi/sorting-algorithms-quicksort-explained-b5bb814df35b